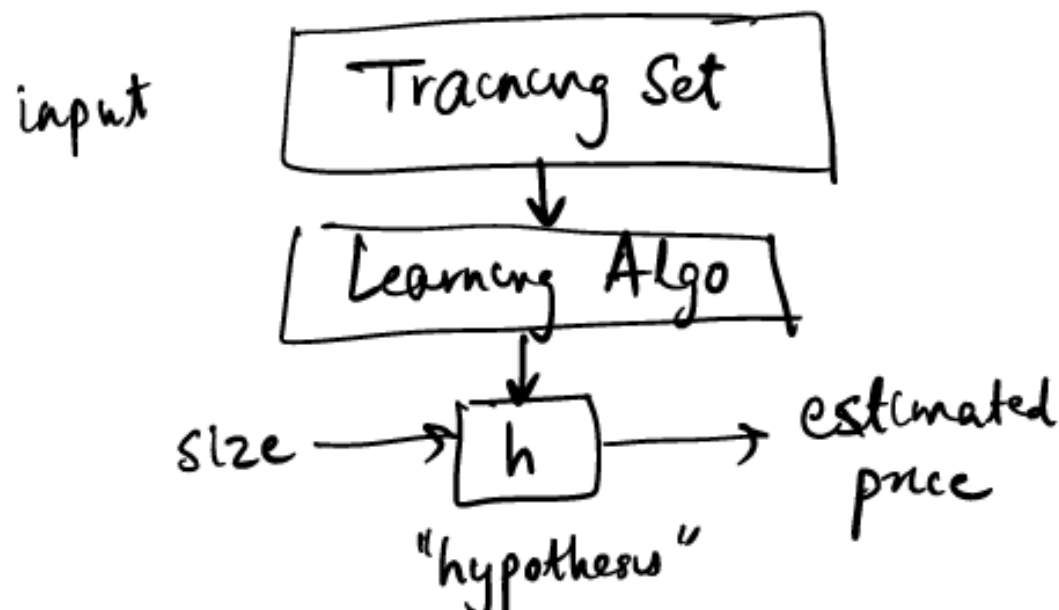


Linear Regression & Gradient Descent

Housing dataset

Size	Price (\$ 1000s)
2104	400
1416	232
1534	315



How to represent h ?

$$h(x) = \theta_0 + \theta_1 x$$

More features

	Size	# bedrooms	Price
$x^{(1)}$	2104	4	400
$x^{(2)}$	1416	3	232

$$x_1^{(1)} = 2104$$

$$x_1^{(2)} = 1416$$

$$x_1 = \text{size}, \quad x_2 = \text{\# bedrooms}$$

$$h(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

$$h(x) = \sum_{j=0}^2 \theta_j x_j$$

$$\text{Define } x_0 = 1$$

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \end{bmatrix}$$

parameters.

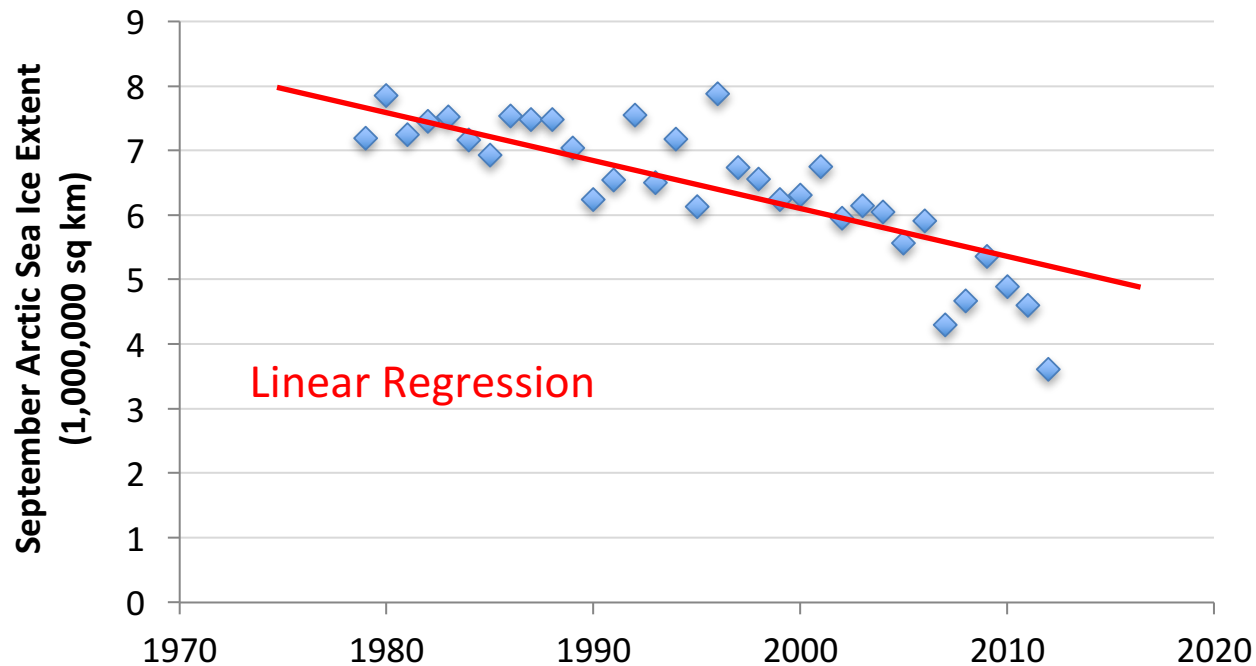
$$x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix}$$

always 1
size
bedrooms

Regression

Given:

- Data $\mathbf{X} = \{x^{(1)}, \dots, x^{(n)}\}$ where $x^{(i)} \in \mathbb{R}^d$
- Corresponding labels $\mathbf{y} = \{y^{(1)}, \dots, y^{(n)}\}$ where $y^{(i)} \in \mathbb{R}$



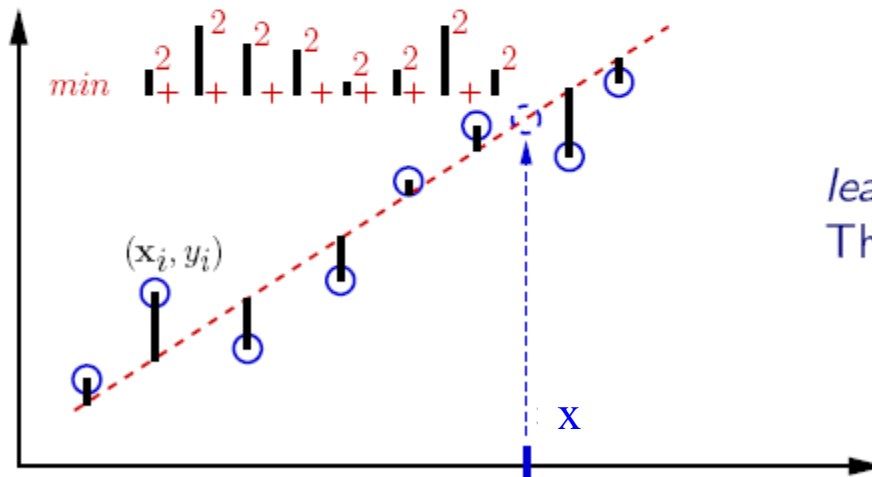
Linear Regression

- Hypothesis:

$$y = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_d x_d = \sum_{j=0}^d \theta_j x_j$$

Assume $x_0 = 1$

- Fit model by minimizing sum of squared errors



least squares (LSQ)
The fitted line is used as a predictor

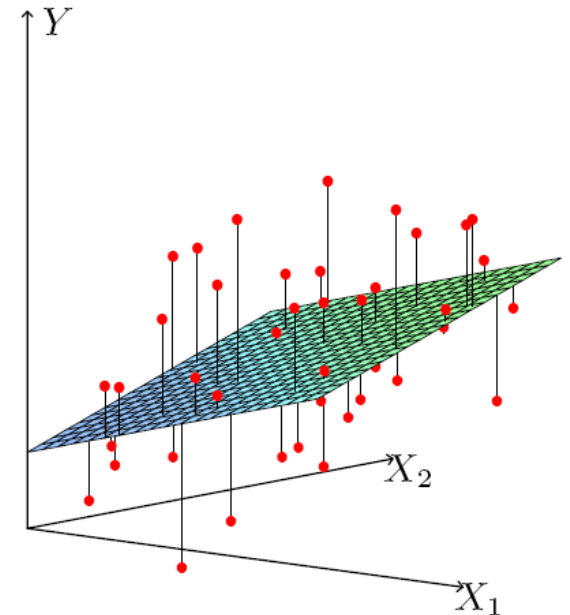
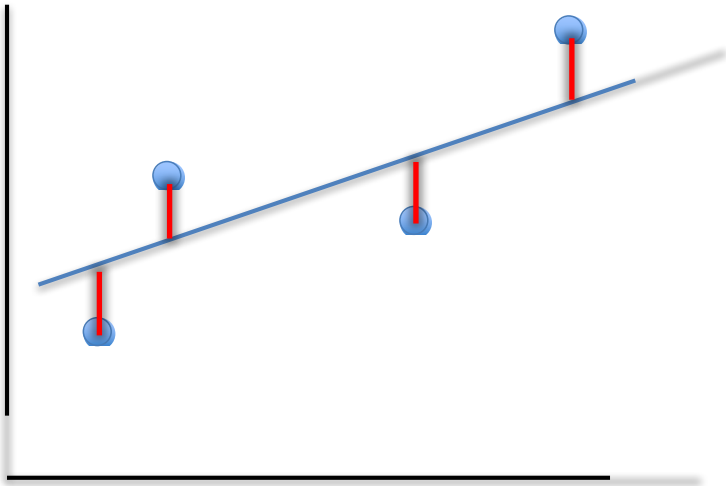
Why Square here ??

Least Squares Linear Regression

- Cost Function

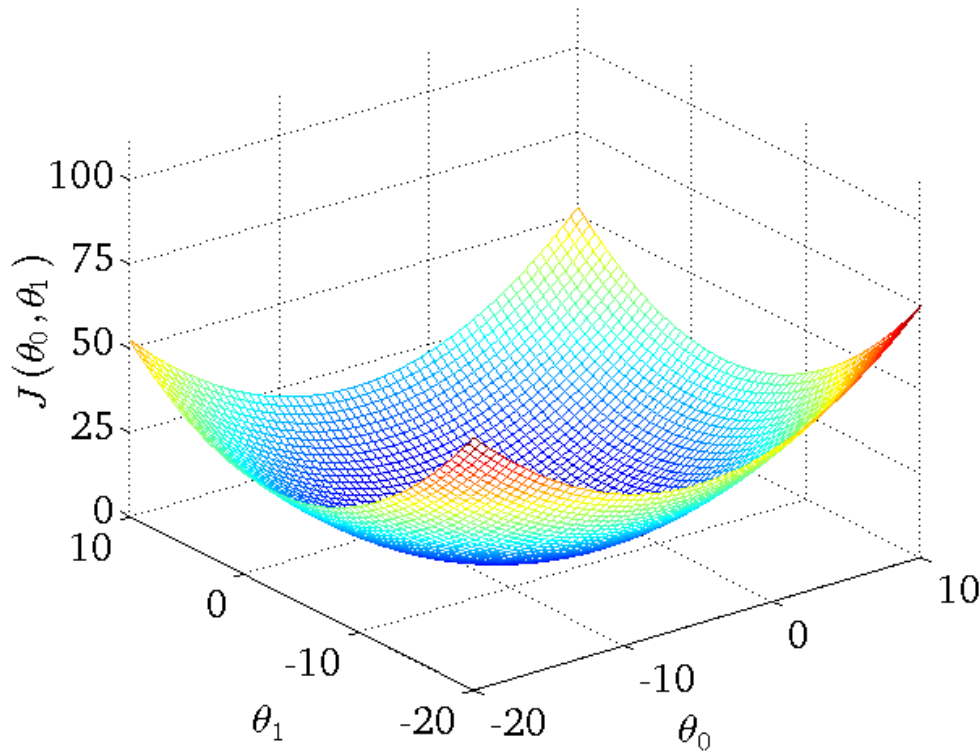
$$J(\theta) = \frac{1}{2n} \sum_{i=1}^n \left(h_{\theta} \left(\mathbf{x}^{(i)} \right) - y^{(i)} \right)^2$$

- Fit by solving $\min_{\theta} J(\theta)$



Dividing by n , the number of data points, is a way to compute the average error. By multiplying by $1/2$, you effectively calculate half of the average squared error, which aligns with the concept of minimizing the mean squared error.

Intuition Behind Cost Function



Think of the cost function as a surface in a multi-dimensional parameter space.

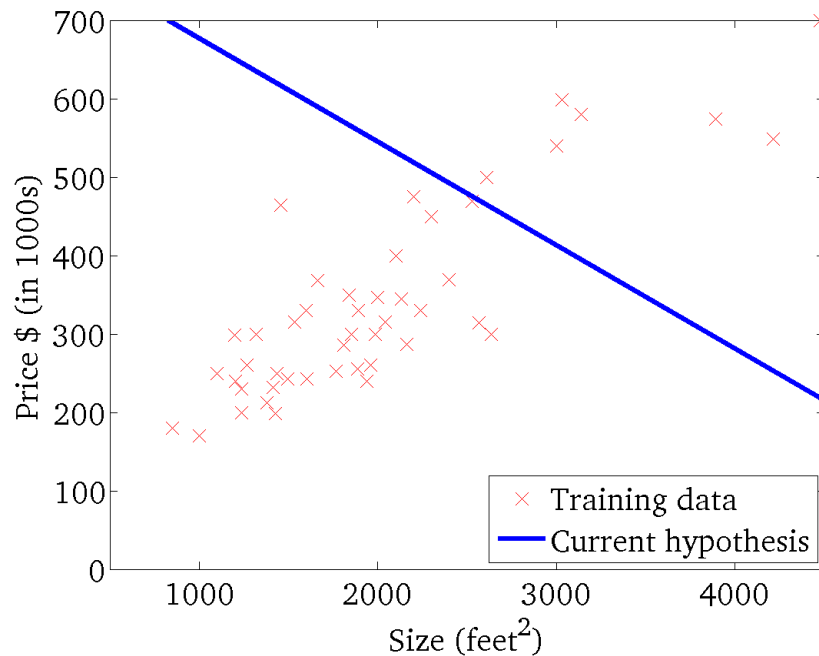
Each point on this surface represents a set of model parameters, and the elevation at that point corresponds to the cost.

The goal is to find the lowest point on this surface, which represents the best-fitting model.

Intuition Behind Cost Function

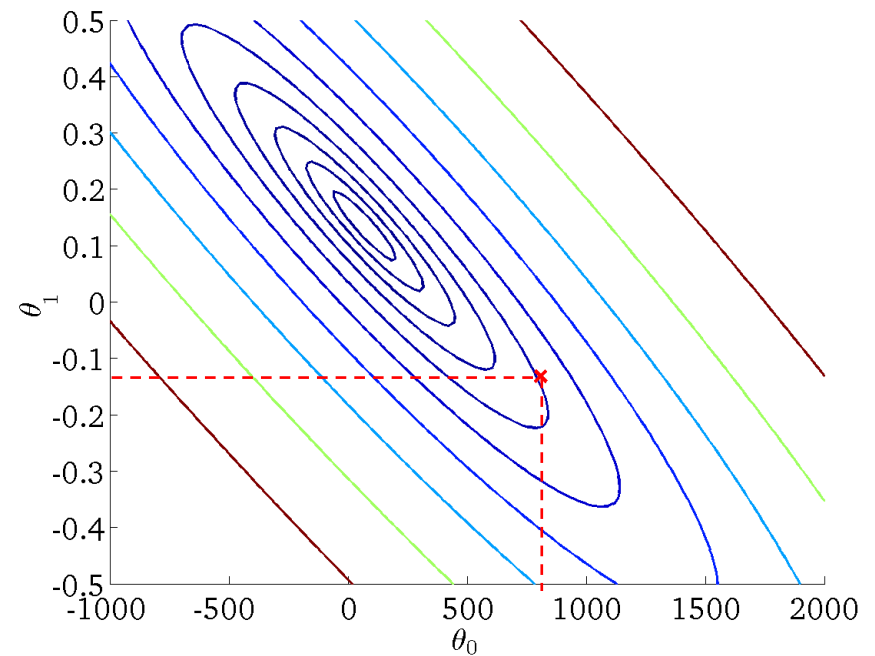
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

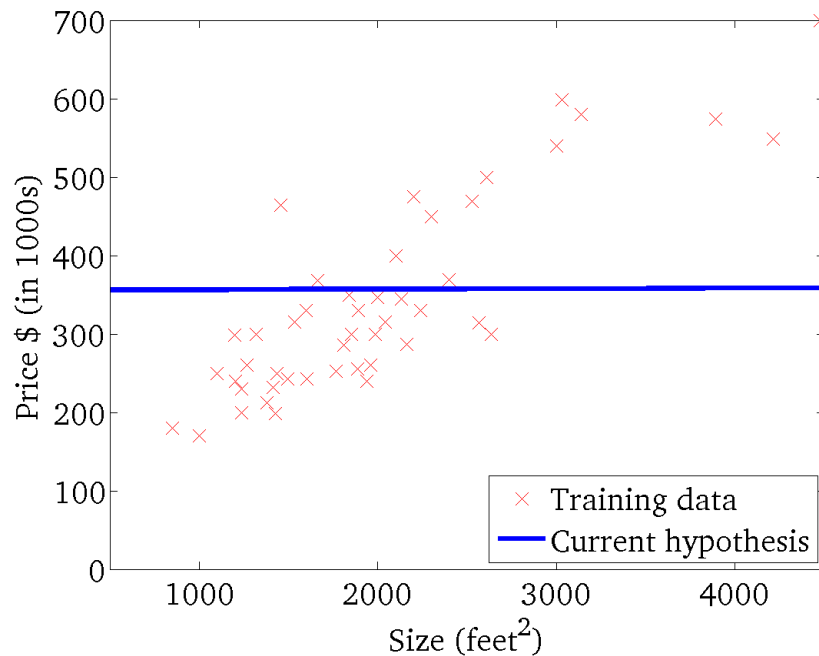
(function of the parameters θ_0, θ_1)



Intuition Behind Cost Function

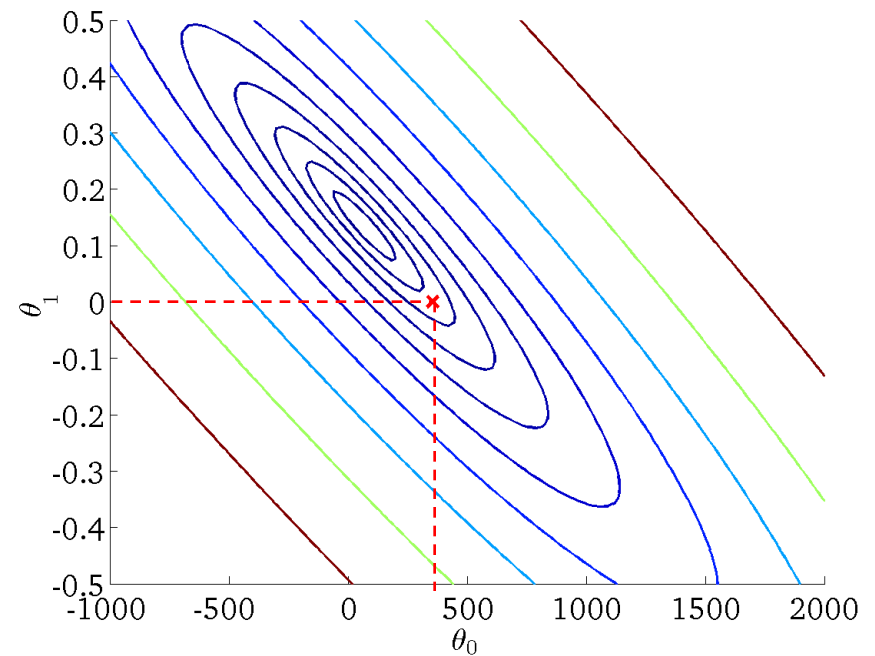
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

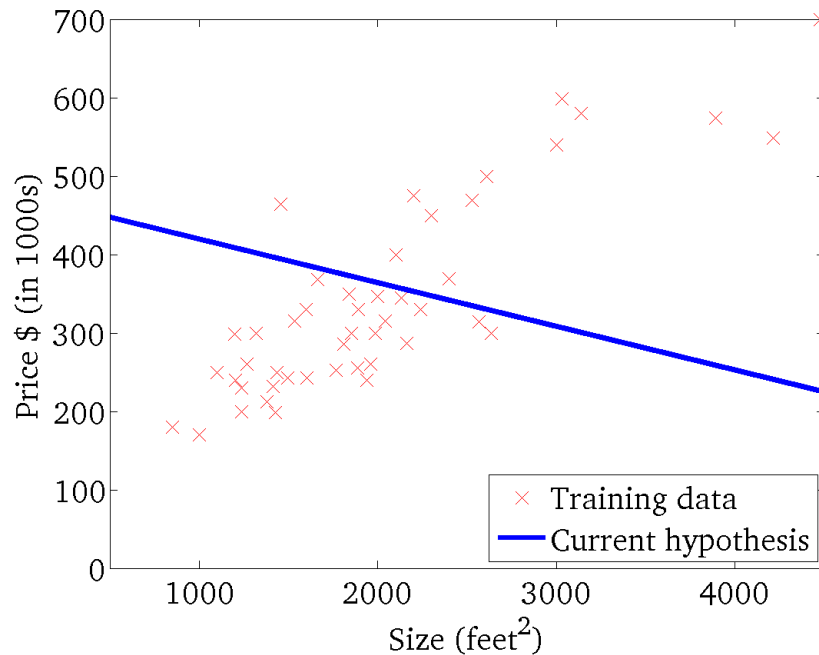
(function of the parameters θ_0, θ_1)



Intuition Behind Cost Function

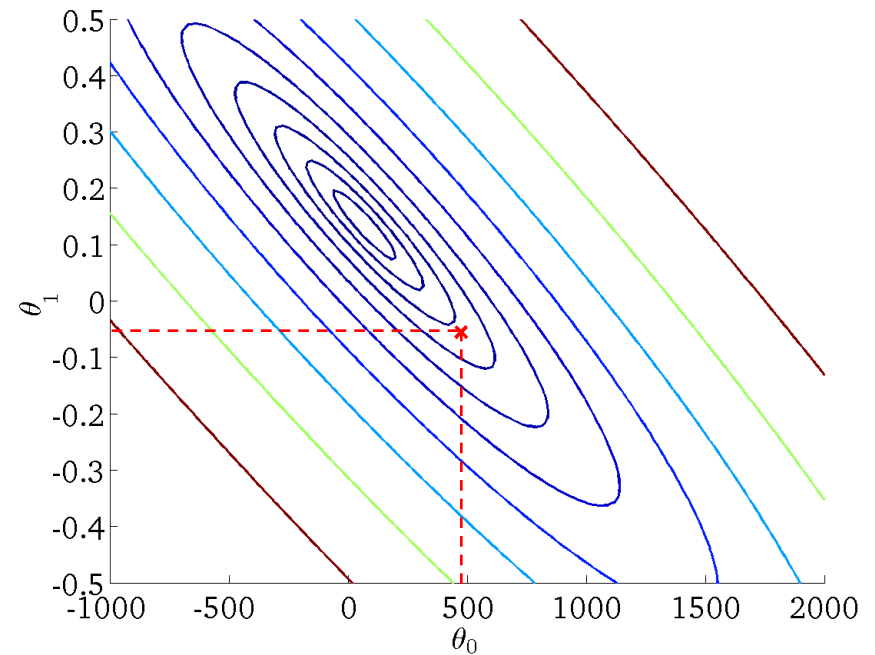
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

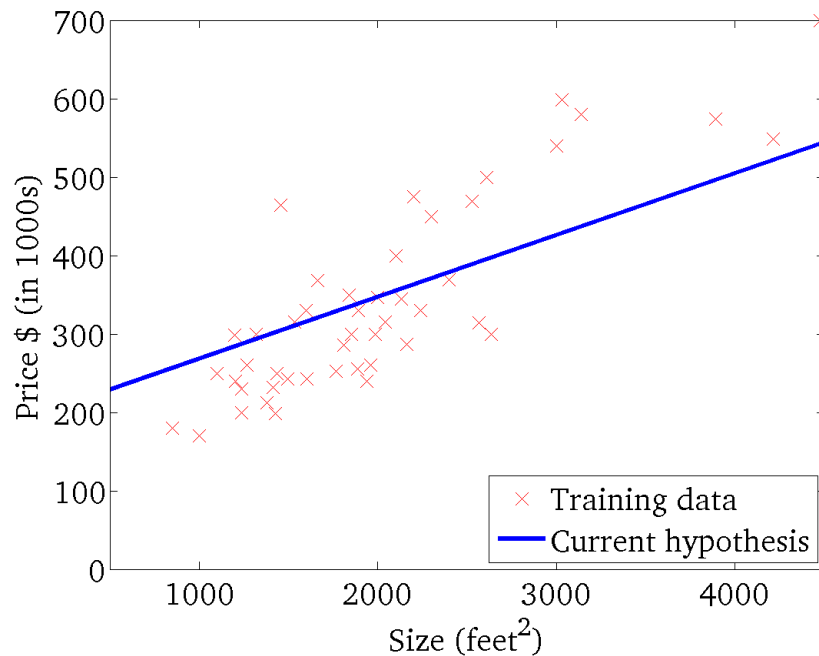
(function of the parameters θ_0, θ_1)



Intuition Behind Cost Function

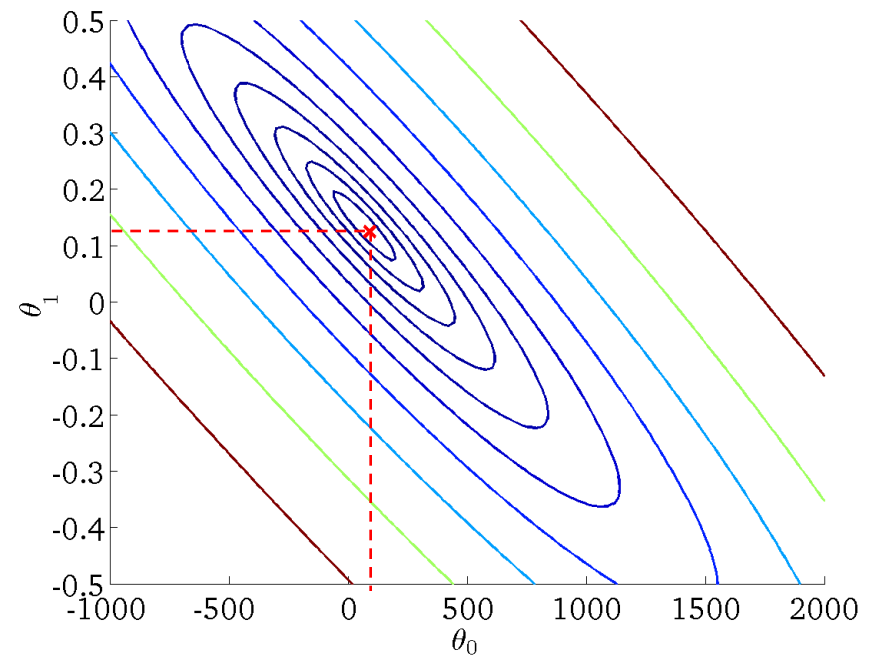
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



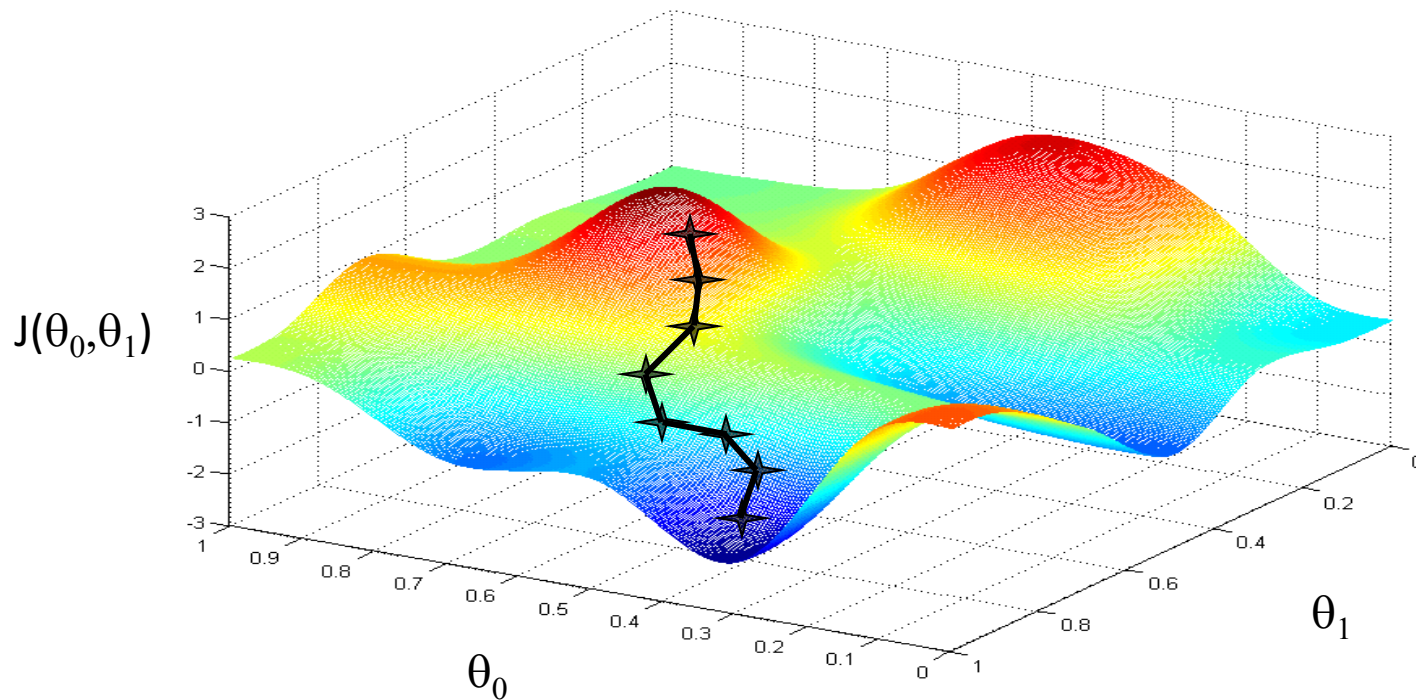
$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)



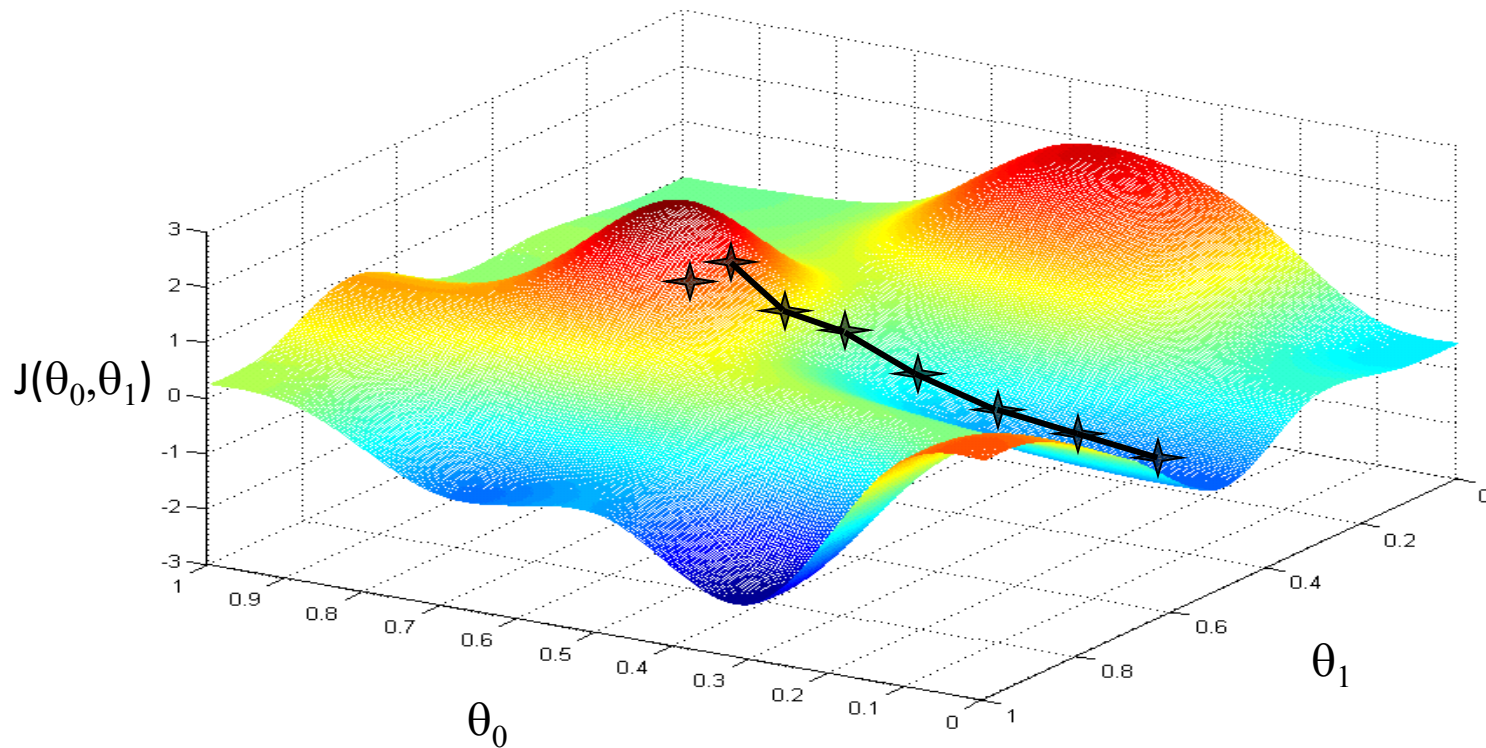
Basic Search Procedure

- Choose initial value for θ
- Until we reach a minimum:
 - Choose a new value for θ to reduce $J(\theta)$



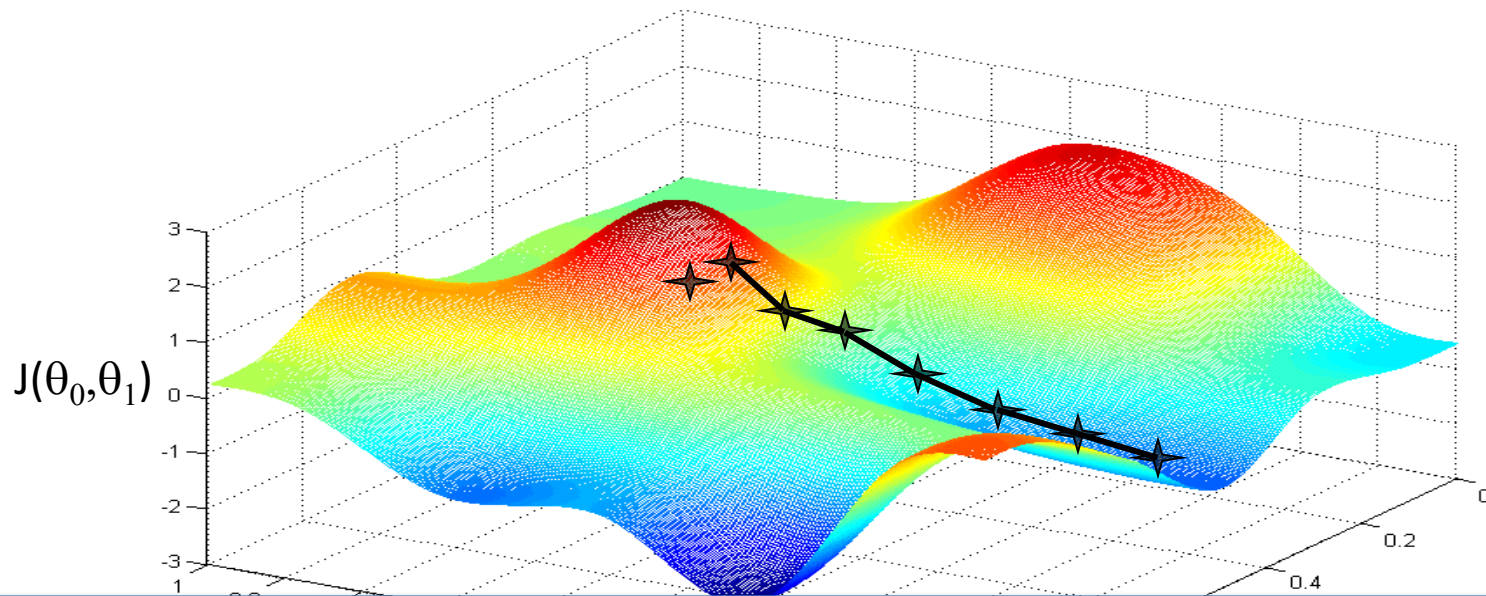
Basic Search Procedure

- Choose initial value for θ
- Until we reach a minimum:
 - Choose a new value for θ to reduce $J(\theta)$
 - How to find the minimum of a function (mathematically) ?
 -



Basic Search Procedure

- Choose initial value for θ
- Until we reach a minimum:
 - Choose a new value for θ to reduce $J(\theta)$
 - How to find the minimum of a function (mathematically) ?



Since the least squares objective function is convex (concave), we don't need to worry about local minima in linear regression

Gradient

Derivative vs Gradient ?

- Derivative gives you the rate of change of the function at a specific point.
- The gradient points in the direction of the steepest increase of the function. Its magnitude indicates the rate of change in that direction.

$$\nabla_{\theta} J(\theta)^T = \begin{bmatrix} \frac{\partial J}{\partial \theta_0} \\ \frac{\partial J}{\partial \theta_1} \\ \vdots \end{bmatrix}$$

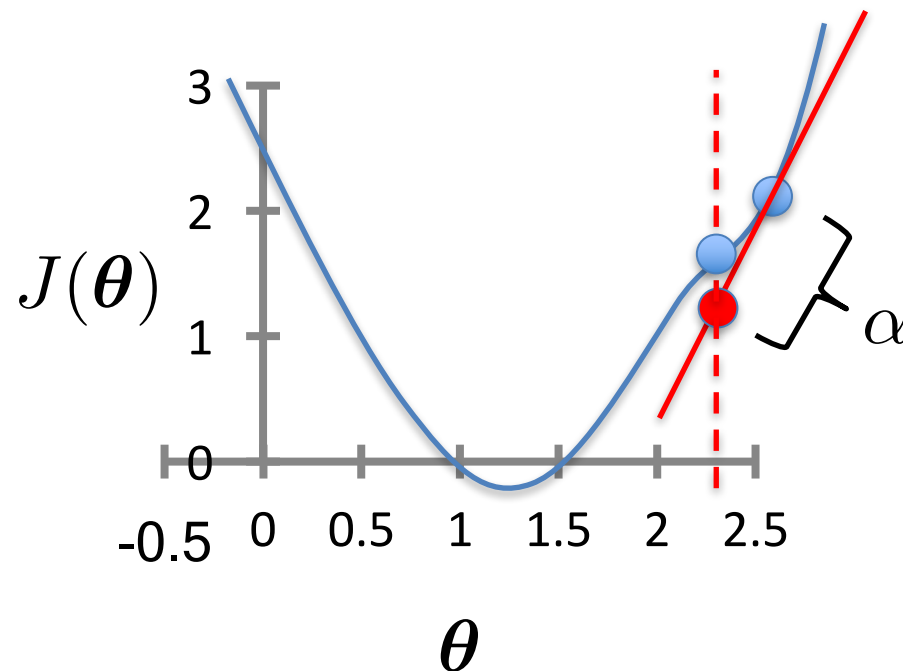
Gradient Descent

- Initialize θ
- Repeat until convergence

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

simultaneous update
for $j = 0 \dots d$

learning rate (small)
e.g., $\alpha = 0.05$



Gradient Descent

- Initialize θ
- Repeat until convergence

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

simultaneous update
for $j = 0 \dots d$

For Linear Regression:

$$\begin{aligned} \frac{\partial}{\partial \theta_j} J(\theta) &= \frac{\partial}{\partial \theta_j} \frac{1}{2n} \sum_{i=1}^n \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2 \\ &= \frac{1}{n} \sum_{i=1}^n (h_{\theta}(x^{(i)}) - y^{(i)}) \frac{\partial}{\partial \theta_j} (h_{\theta}(x^{(i)}) - y^{(i)}) \\ &\quad \text{(chain rule)} \\ &= \frac{1}{n} \sum_{i=1}^n (h_{\theta}(x^{(i)}) - y^{(i)}) \frac{\partial}{\partial \theta_j} (\theta_0 x_0 + \theta_1 x_1 + \dots + \theta_d x_d - y^{(i)}) \\ &= \frac{1}{n} \sum_{i=1}^n (h_{\theta}(x^{(i)}) - y^{(i)}) x_j \\ \theta_j &:= \theta_j - \alpha \frac{1}{n} \sum_{i=1}^n (h_{\theta}(x^{(i)}) - y^{(i)}) x_j \\ \theta_j &:= \theta_j - \alpha \sum_{i=1}^n (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)} \end{aligned}$$

Gradient Descent for Linear Regression

- Initialize θ
- Repeat until convergence

$$\theta_j \leftarrow \theta_j - \alpha \frac{1}{n} \sum_{i=1}^n \left(h_{\theta} \left(\mathbf{x}^{(i)} \right) - y^{(i)} \right) x_j^{(i)} \quad \begin{array}{l} \text{simultaneous} \\ \text{update} \\ \text{for } j = 0 \dots d \end{array}$$

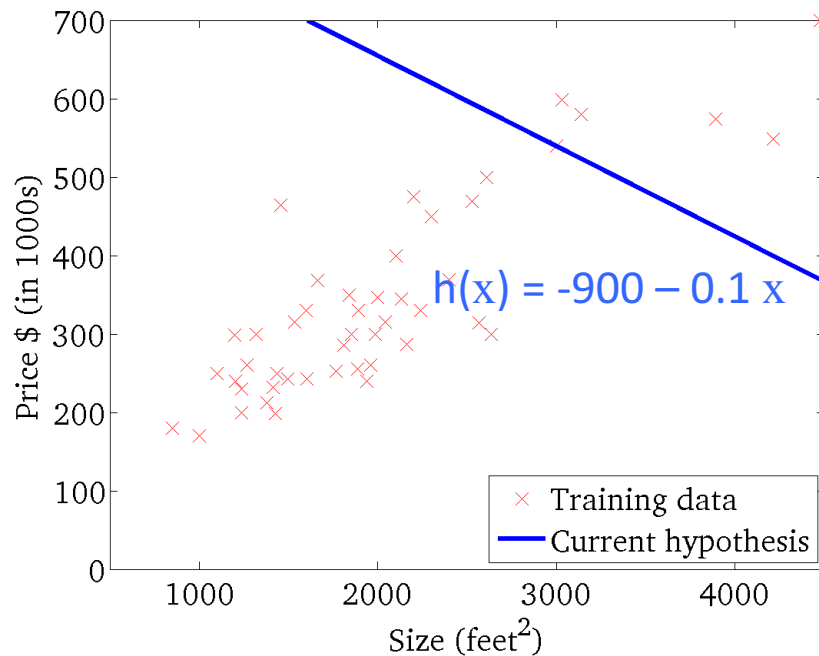
- To achieve simultaneous update
 - At the start of each GD iteration, compute $h_{\theta} \left(\mathbf{x}^{(i)} \right)$
 - Use this stored value in the update step loop
- Assume convergence when $\|\theta_{new} - \theta_{old}\|_2 < \epsilon$

$$\text{L}_2 \text{ norm: } \quad \|\mathbf{v}\|_2 = \sqrt{\sum_i v_i^2} = \sqrt{v_1^2 + v_2^2 + \dots + v_{|\mathbf{v}|}^2}$$

Gradient Descent

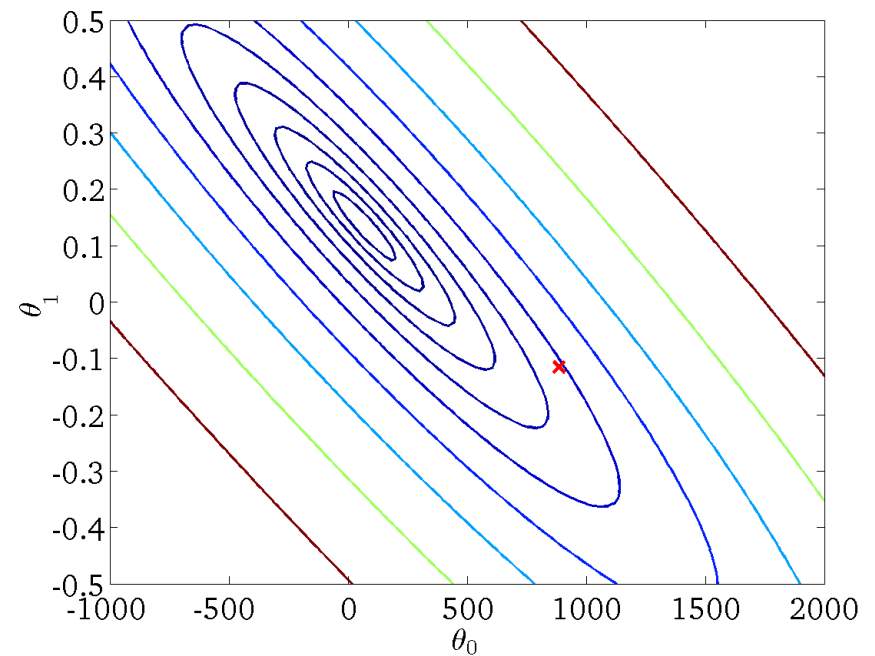
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

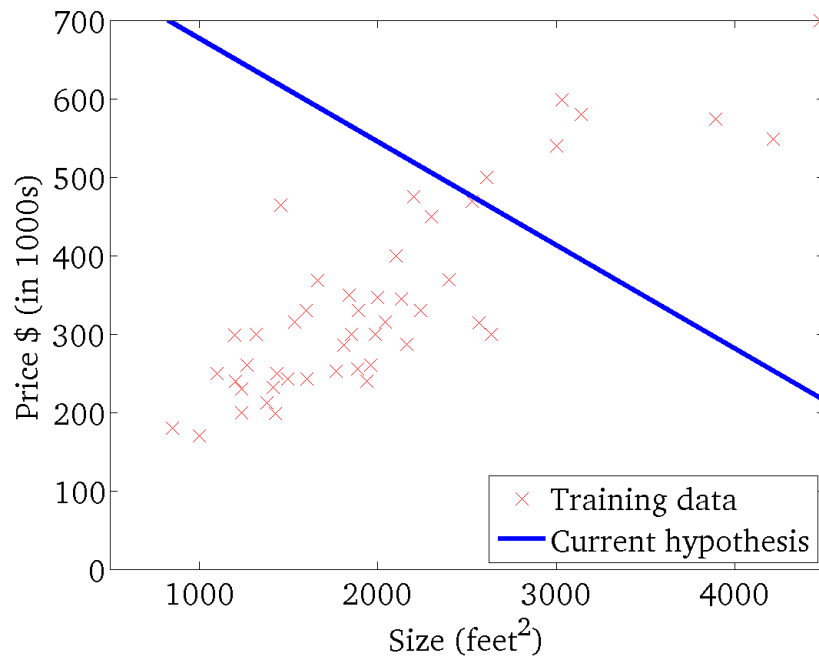
(function of the parameters θ_0, θ_1)



Gradient Descent

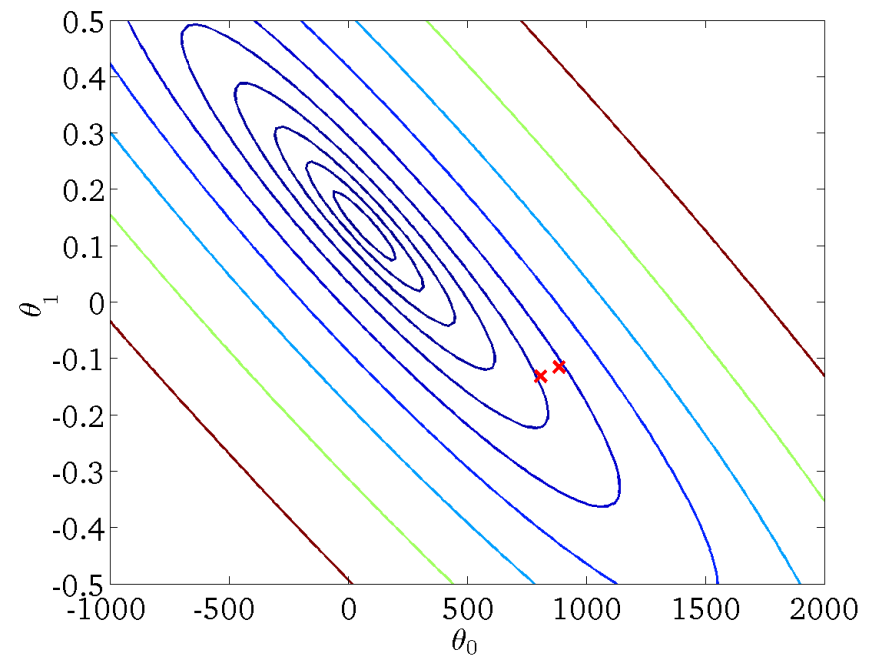
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

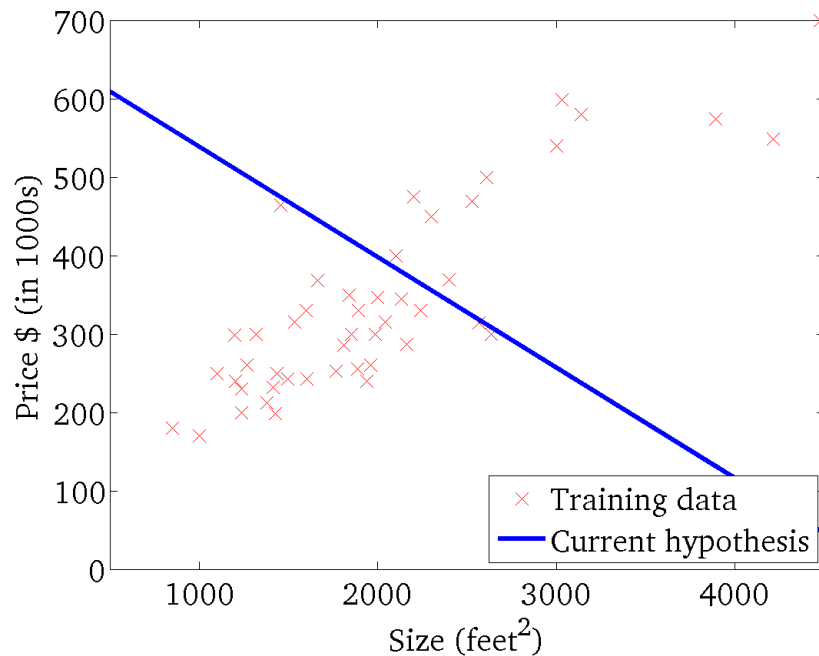
(function of the parameters θ_0, θ_1)



Gradient Descent

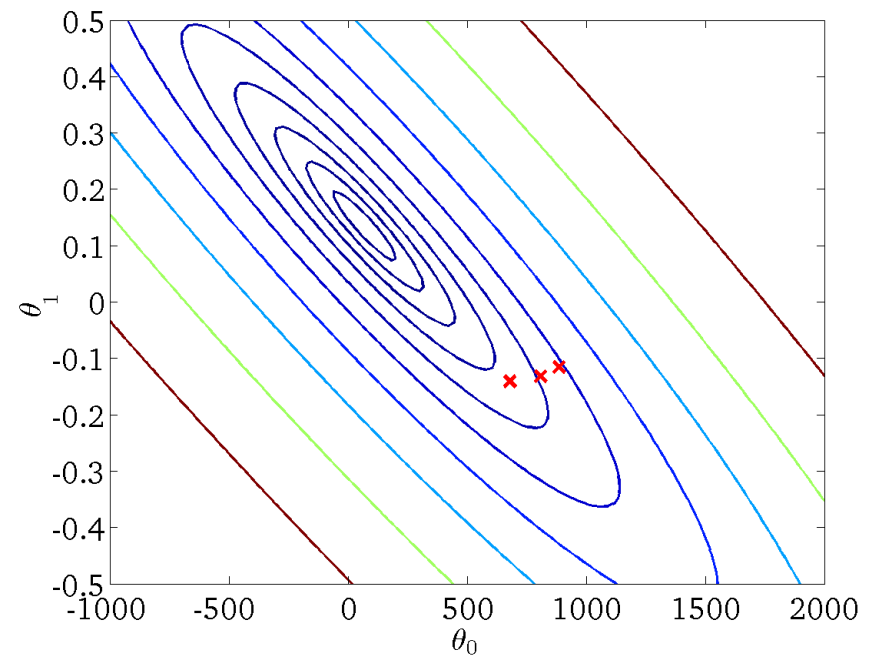
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

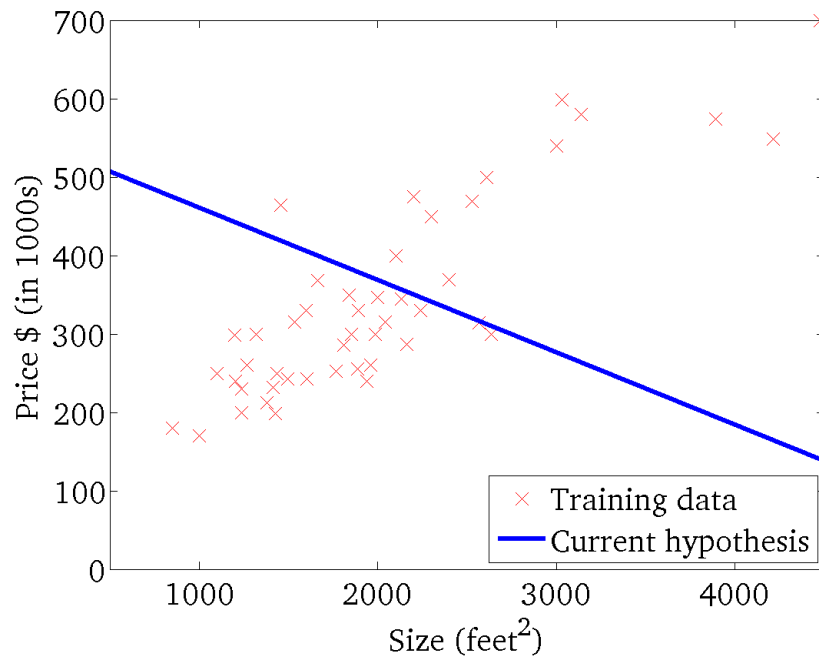
(function of the parameters θ_0, θ_1)



Gradient Descent

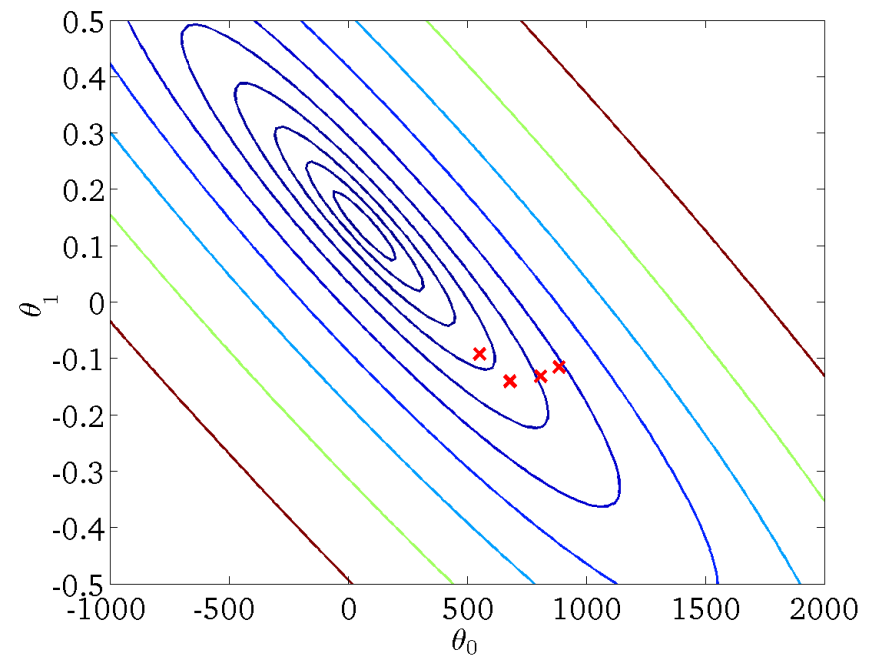
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

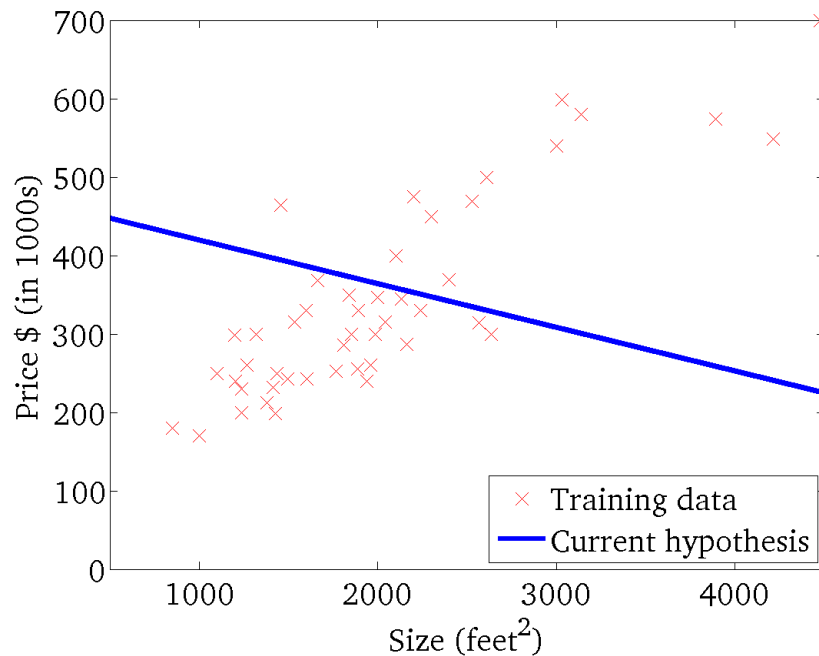
(function of the parameters θ_0, θ_1)



Gradient Descent

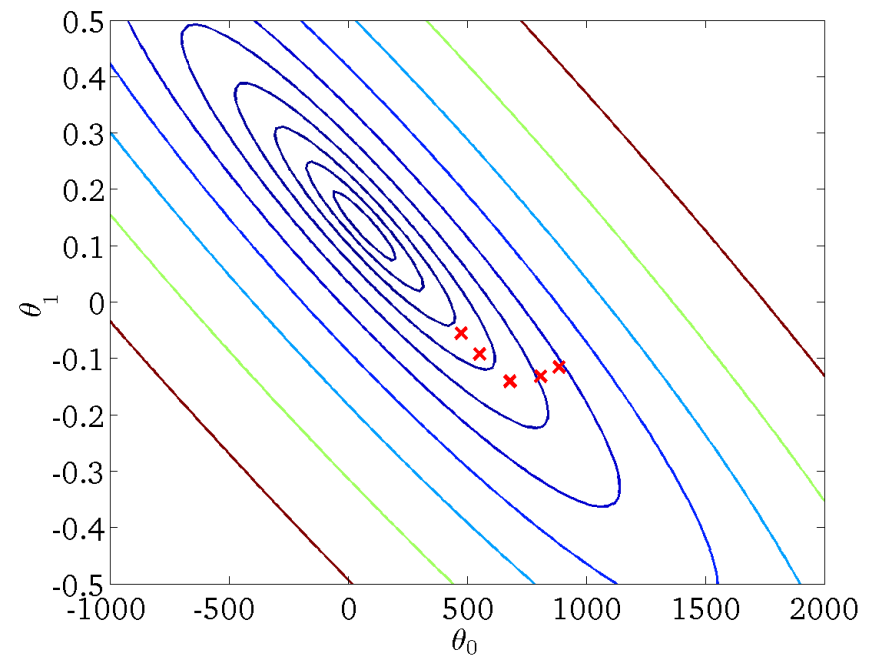
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

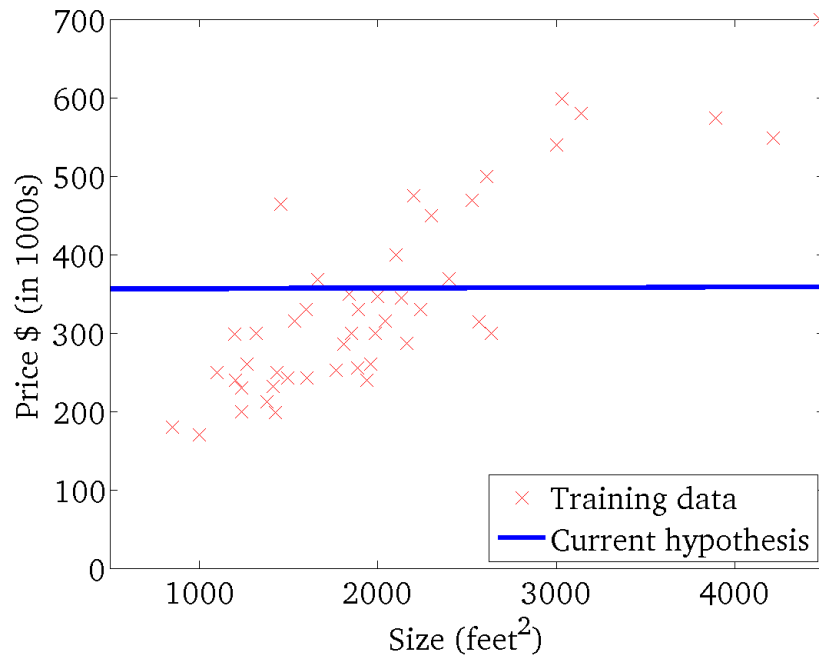
(function of the parameters θ_0, θ_1)



Gradient Descent

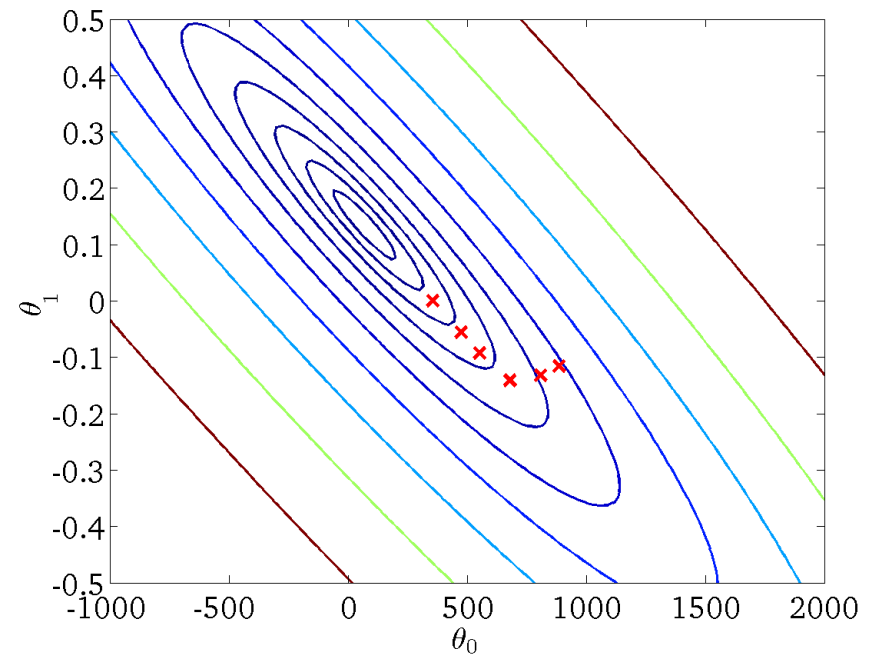
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

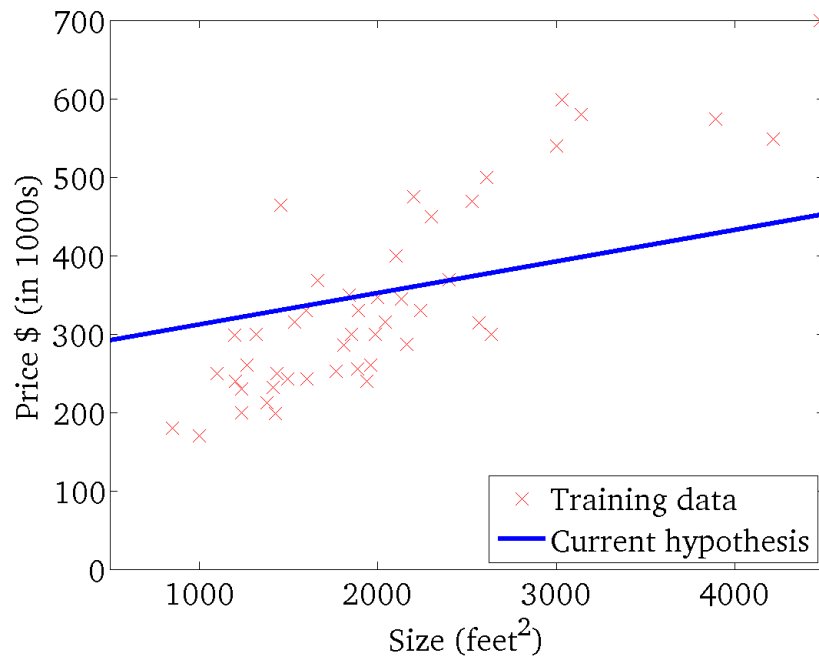
(function of the parameters θ_0, θ_1)



Gradient Descent

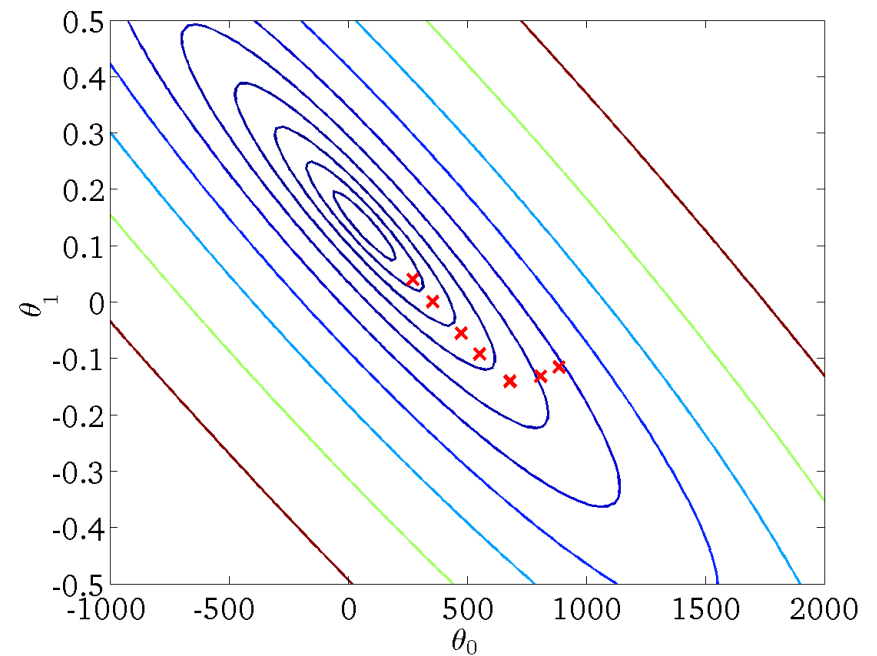
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

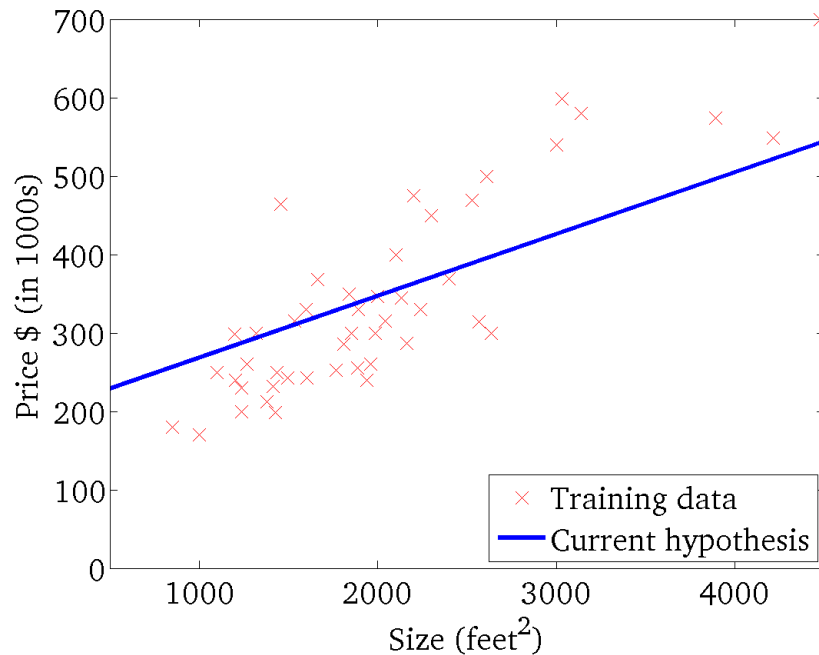
(function of the parameters θ_0, θ_1)



Gradient Descent

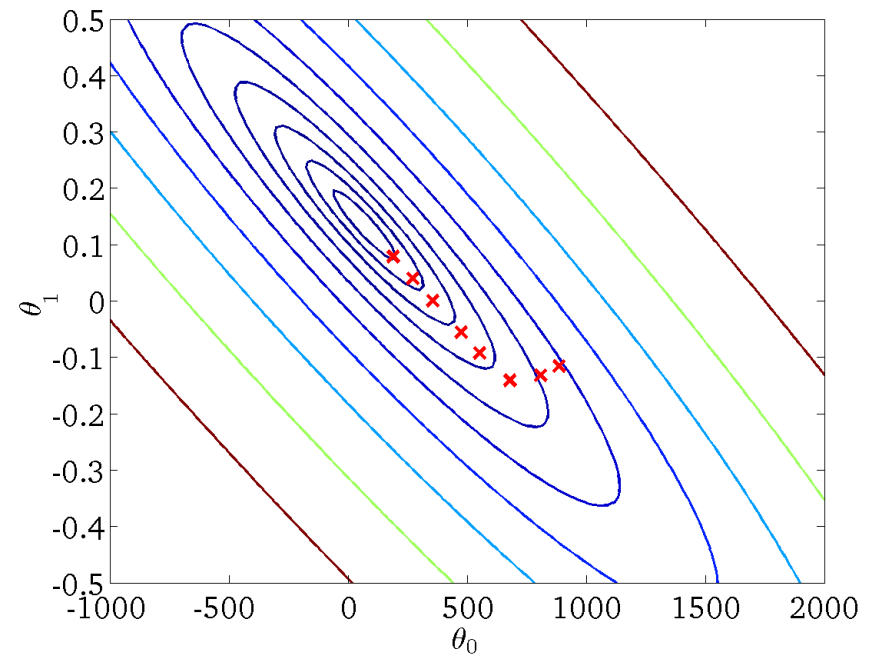
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

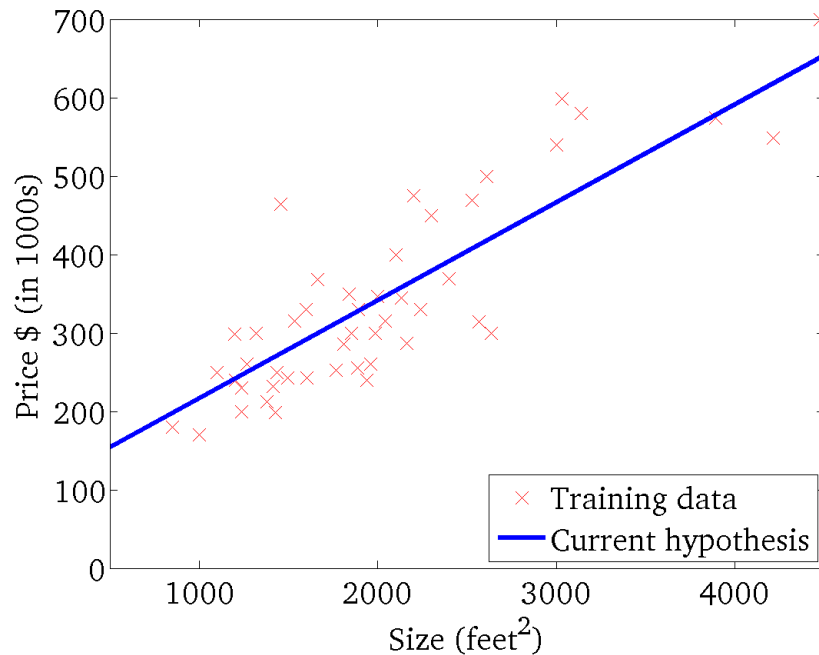
(function of the parameters θ_0, θ_1)



Gradient Descent

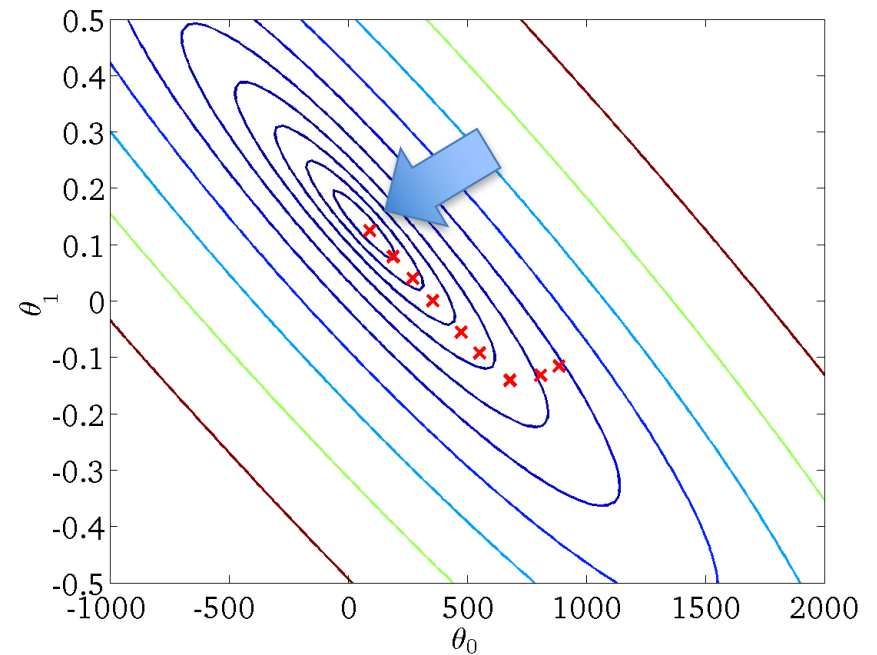
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



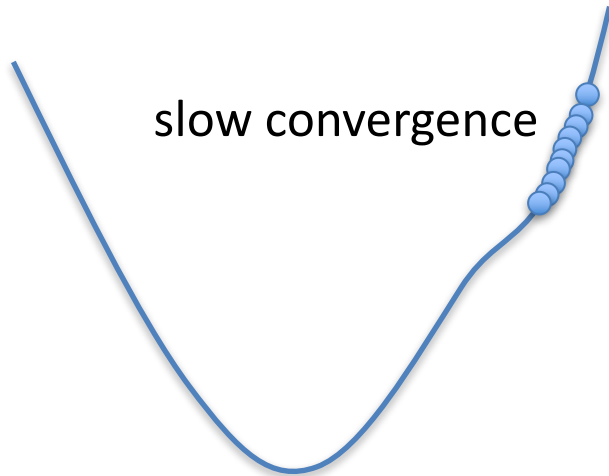
$$J(\theta_0, \theta_1)$$

(function of the parameters θ_0, θ_1)

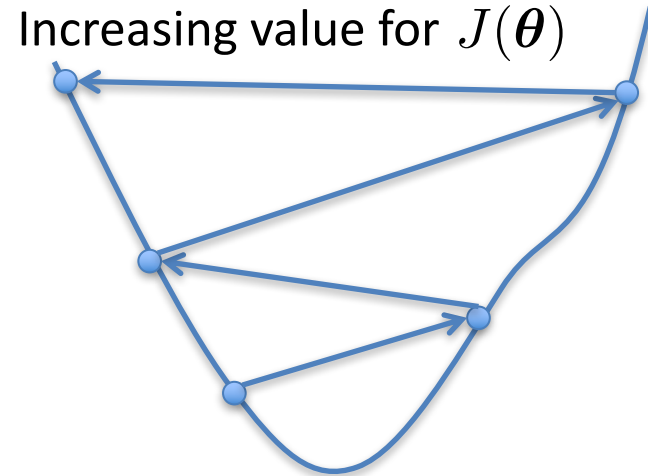


Choosing α

α too small



α too large



- May overshoot the minimum
- May fail to converge
- May even diverge

To see if gradient descent is working, print out $J(\theta)$ each iteration

- The value should decrease at each iteration
- If it doesn't, adjust α

Stochastic Gradient Descent
(SGD)

Batch Gradient Descent

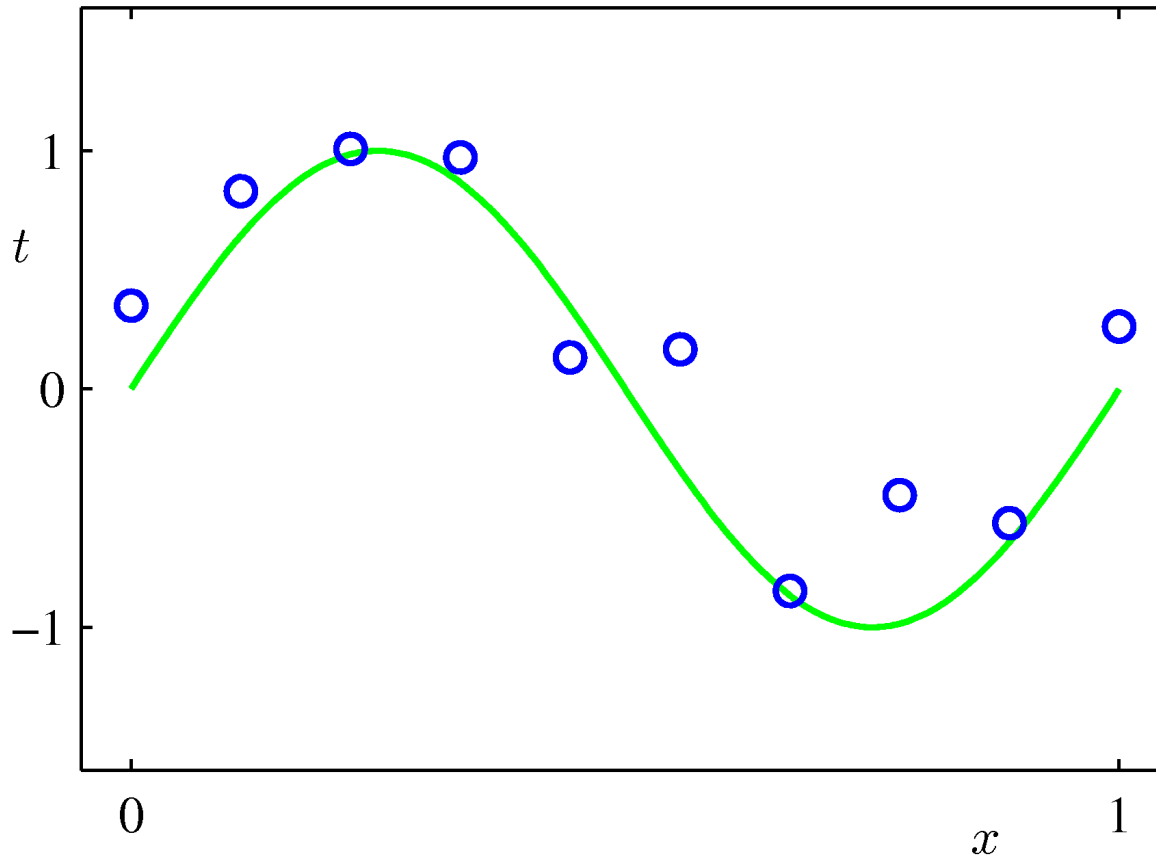
Mini-Batch Gradient Descent

Extending Linear Regression to More Complex Models

- The inputs $\mathbf{X}$ for linear regression can be:
 - Original quantitative inputs
 - Transformation of quantitative inputs
 - e.g. log, exp, square root, square, etc.
 - Polynomial transformation
 - example: $y = \beta_0 + \beta_1 \cdot x + \beta_2 \cdot x^2 + \beta_3 \cdot x^3$
 - Basis expansions
 - Interactions between variables
 - example: $x_3 = x_1 \cdot x_2$

This allows use of **linear** regression techniques to fit **non-linear** datasets.

Example of Fitting a Polynomial Curve with a Linear Model



$$y = \theta_0 + \theta_1 x + \theta_2 x^2 + \dots + \theta_p x^p = \sum_{j=0}^p \theta_j x^j$$

Linear Basis Function Models

- Generally,

$$h_{\theta}(\mathbf{x}) = \sum_{j=0}^d \theta_j \underbrace{\phi_j(\mathbf{x})}_{\text{basis function}}$$

- Typically, $\phi_0(\mathbf{x}) = 1$ so that θ_0 acts as a bias
- In the simplest case, we use linear basis functions :

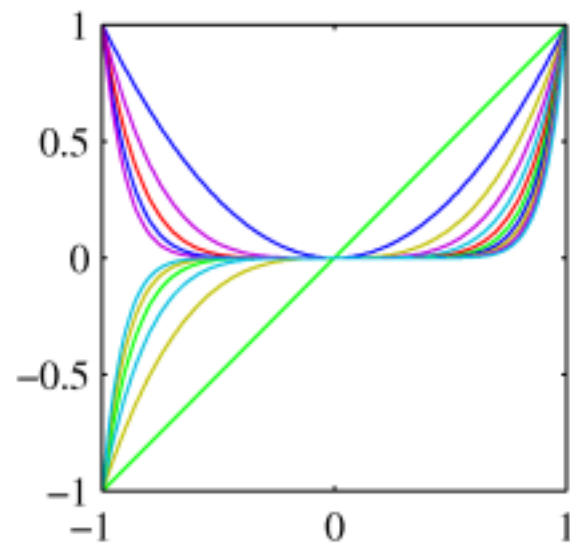
$$\phi_j(\mathbf{x}) = x_j$$

Linear Basis Function Models

- Polynomial basis functions:

$$\phi_j(x) = x^j$$

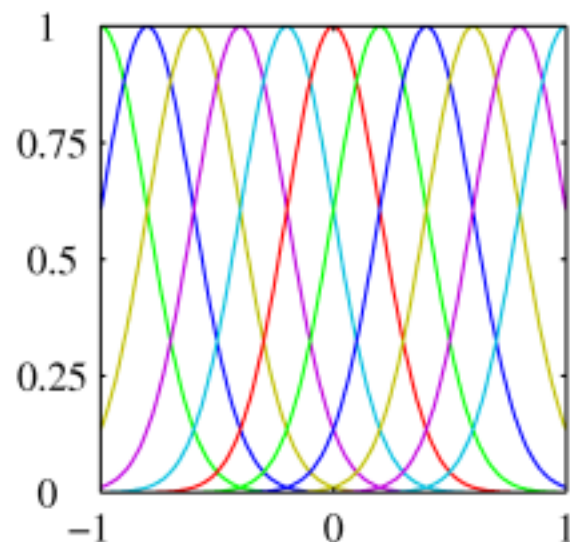
- These are global; a small change in x affects all basis functions



- Gaussian basis functions:

$$\phi_j(x) = \exp \left\{ -\frac{(x - \mu_j)^2}{2s^2} \right\}$$

- These are local; a small change in x only affect nearby basis functions. μ_j and s control location and scale (width).



Linear Basis Function Models

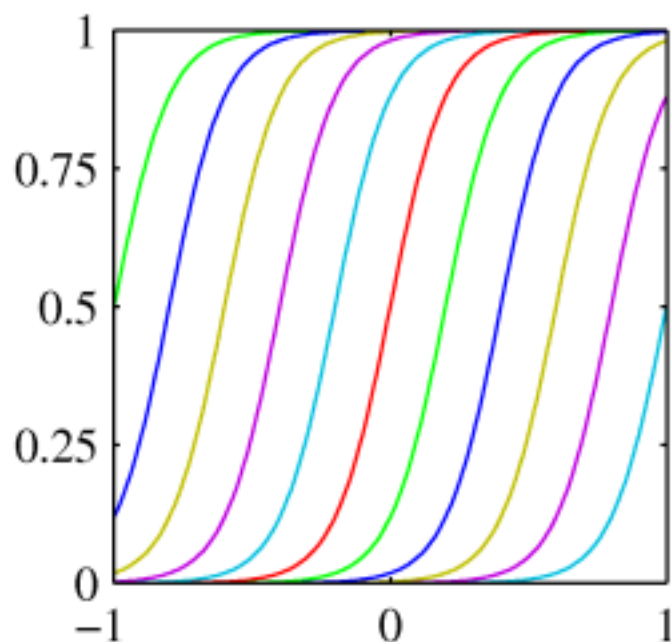
- Sigmoidal basis functions:

$$\phi_j(x) = \sigma \left(\frac{x - \mu_j}{s} \right)$$

where

$$\sigma(a) = \frac{1}{1 + \exp(-a)}$$

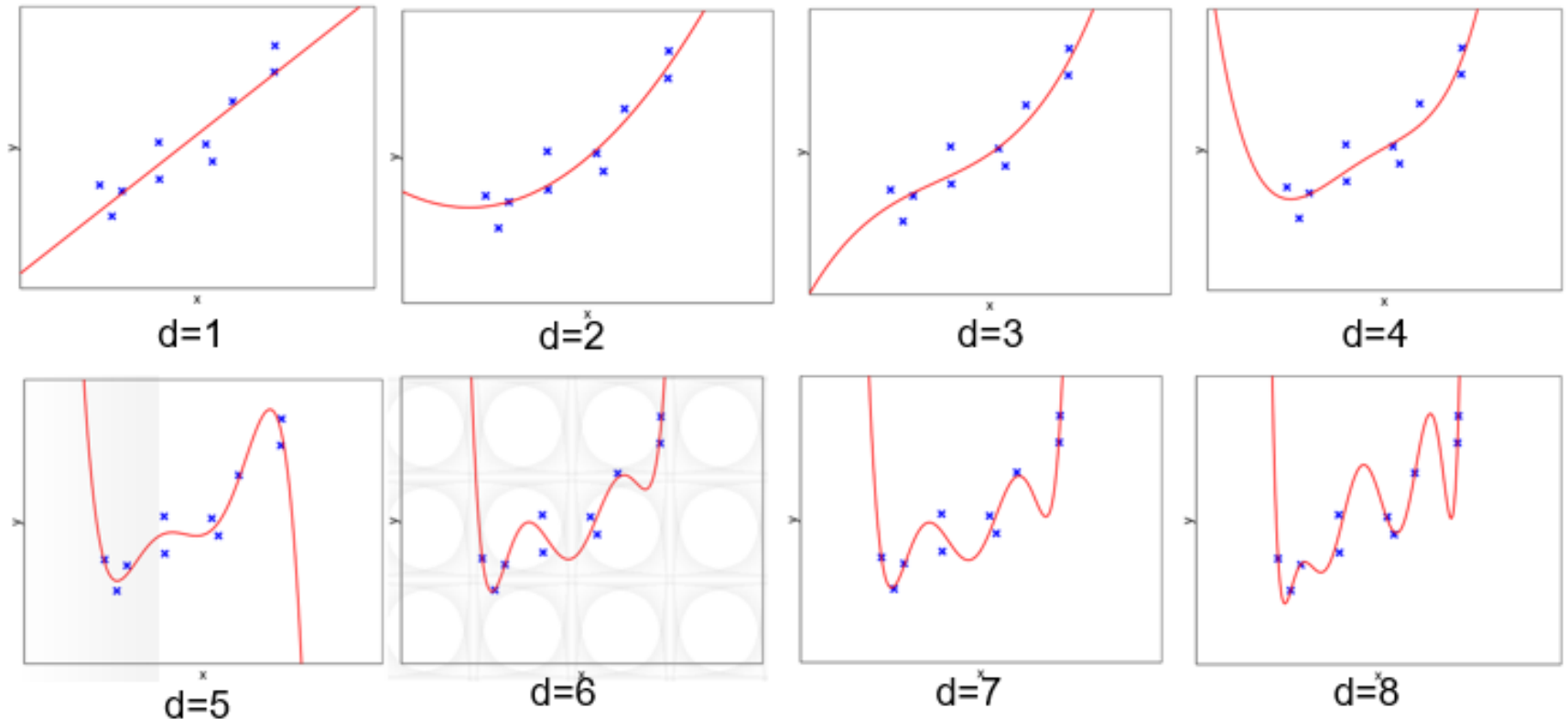
- These are also local; a small change in x only affects nearby basis functions. μ_j and s control location and scale (slope).



Over-Fitting

Every hypothesis has a true error measured on all possible data items we could ever encounter .

Since we don't have all possible data, in order to decide what is a good hypothesis, we measure error over the training set.



BIAS

- In making predictions, a difference occurs between prediction values made by the model and actual values, this difference is known as bias error.

Or

- Inability of machine learning algorithms such as Linear Regression to capture the true relationship between the data points.

Low Bias: will closely match the training data set.

High-Bias: A model with a higher bias would not match the data set closely

High Bias causes under fitting in our model.

High bias mainly occurs due to a much simple model. Below are some ways to reduce the high bias:

- Increase the input features as the model is under fitted.
- Decrease the regularization term.
- Use more complex models, such as including some polynomial features.

Variance

- In machine learning, variance refers to how much the model changes when trained on different subsets of the training data.
- Ideally, a model should not vary too much from one training dataset to another, which means the algorithm should be good in understanding the hidden mapping between inputs and output variables.

Low variance means there is a small variation in the prediction of the target function with changes in the training data set.

High variance shows a large variation in the prediction of the target function with changes in the training dataset.

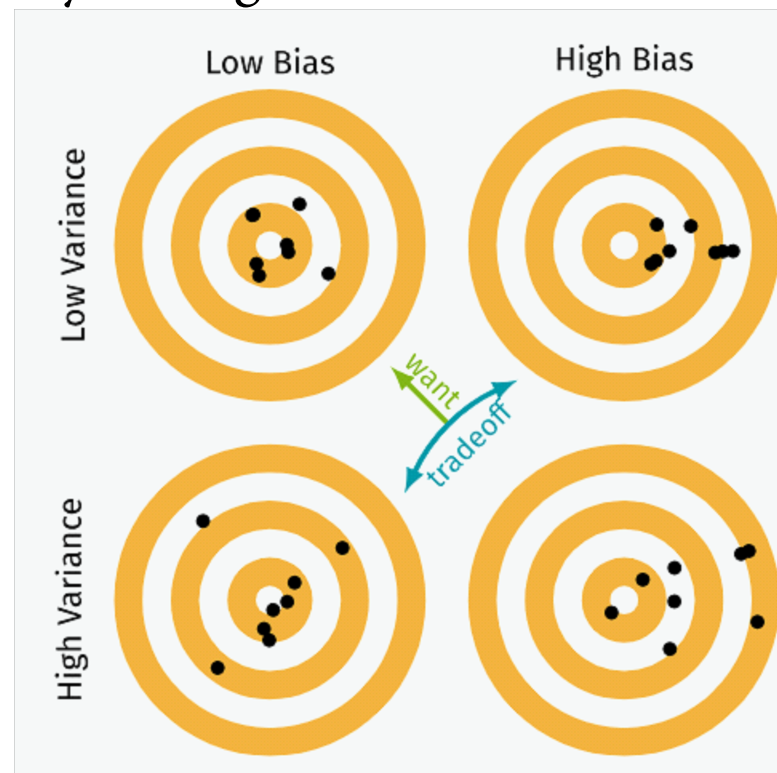
Ways to Reduce High Variance:

Reduce the input features or number of parameters as a model is over fitted.

- Do not use a much complex model.
- Increase the training data.
- Increase the Regularization term.

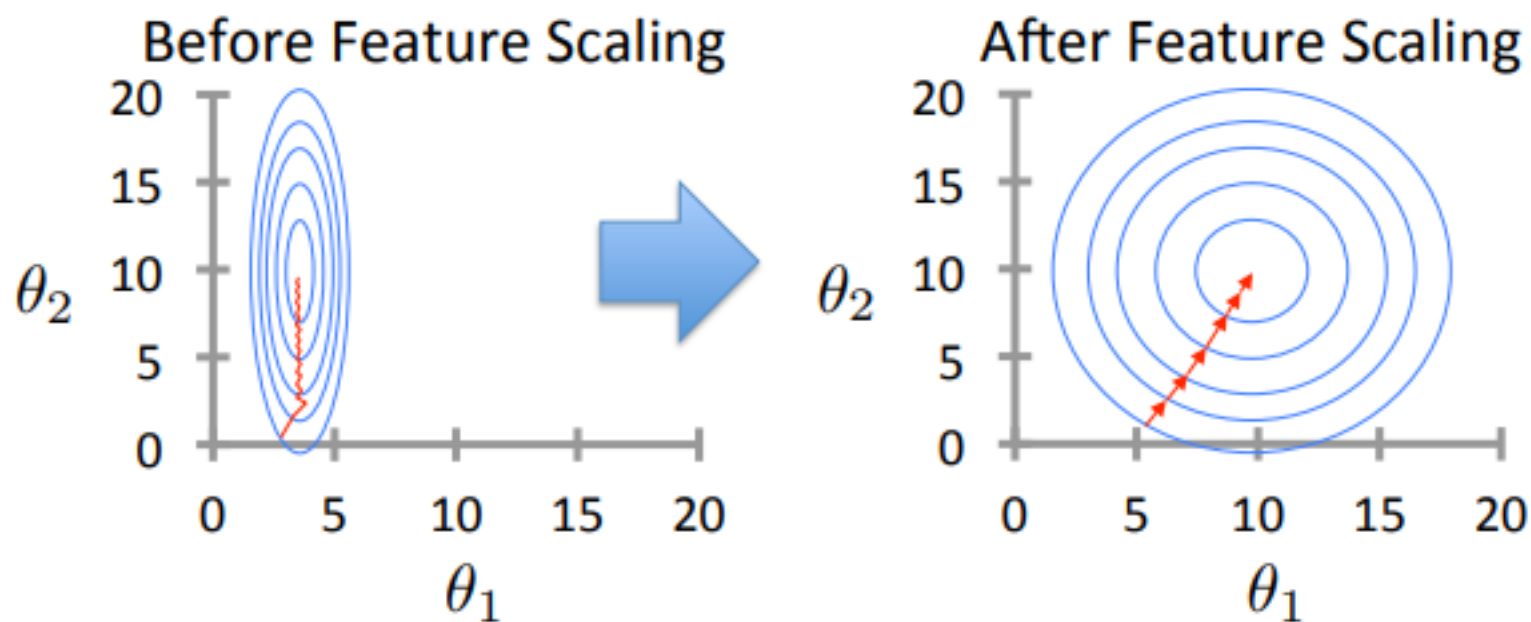
Bias-Variance trade-off

- An optimal model is one in which the model is sensitive to the pattern in our model, but at the same time can generalize to new data.
- This happens when Bias and Variance are both optimal.
- A situation of low bias and high variance is termed as overfitting such that the model fits certainly well with high accuracy on available data and when it sees new data it fails to predict accurately that yield high test error.



Improving Learning: Feature Scaling

- **Idea:** Ensure that feature have similar scales



- Makes gradient descent converge *much* faster

Feature Standardization

- Rescales features to have zero mean and unit variance

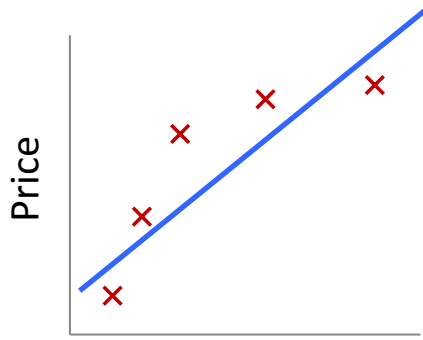
- Let μ_j be the mean of feature j : $\mu_j = \frac{1}{n} \sum_{i=1}^n x_j^{(i)}$

- Replace each value with:

$$x_j^{(i)} \leftarrow \frac{x_j^{(i)} - \mu_j}{s_j} \quad \text{for } j = 1 \dots d \quad (\text{not } x_0!)$$

- s_j is the standard deviation of feature j

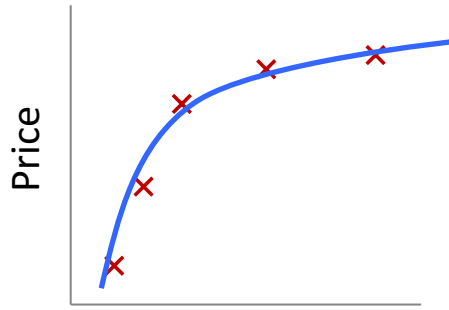
Quality of Fit



Size

$$\theta_0 + \theta_1 x$$

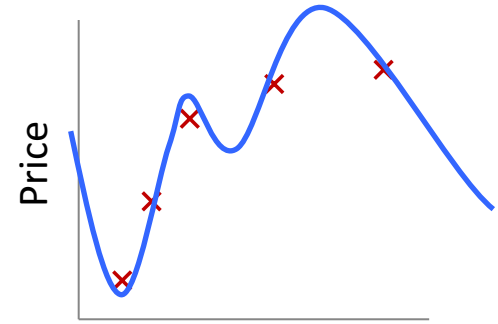
Underfitting
(high bias)



Size

$$\theta_0 + \theta_1 x + \theta_2 x^2$$

Correct fit



Size

$$\theta_0 + \theta_1 x + \theta_2 x^2 + \theta_3 x^3 + \theta_4 x^4$$

Overfitting
(high variance)

Overfitting:

- The learned hypothesis may fit the training set very well ($J(\theta) \approx 0$)
- ...but fails to generalize to new examples

Regularization

- A method for automatically controlling the complexity of the learned hypothesis
- **Idea:** penalize for large values of θ_j
 - Can incorporate into the cost function
 - Works well when we have a lot of features, each that contributes a bit to predicting the label
- Can also address overfitting by eliminating features (either manually or via model selection)

Regularization

- Linear regression objective function

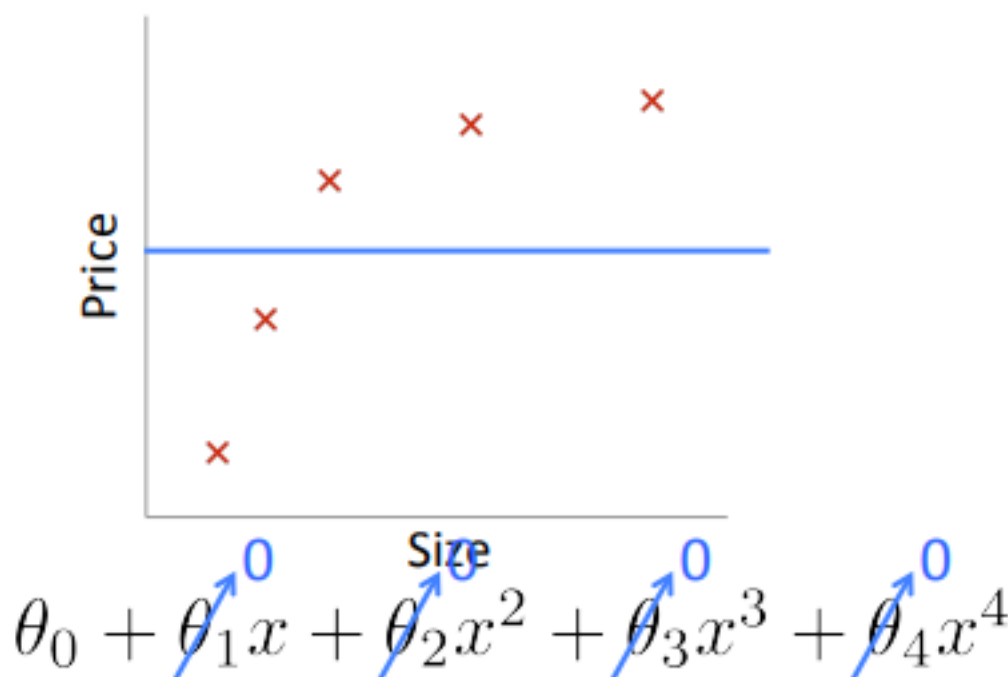
$$J(\boldsymbol{\theta}) = \underbrace{\frac{1}{2n} \sum_{i=1}^n \left(h_{\boldsymbol{\theta}} \left(\mathbf{x}^{(i)} \right) - y^{(i)} \right)^2}_{\text{model fit to data}} + \underbrace{\frac{\lambda}{2} \sum_{j=1}^d \theta_j^2}_{\text{regularization}}$$

- λ is the regularization parameter ($\lambda \geq 0$)
- No regularization on θ_0 !

Understanding Regularization

$$J(\boldsymbol{\theta}) = \frac{1}{2n} \sum_{i=1}^n \left(h_{\boldsymbol{\theta}} \left(\mathbf{x}^{(i)} \right) - y^{(i)} \right)^2 + \lambda \sum_{j=1}^d \theta_j^2$$

- What happens if we set λ to be huge (e.g., 10^{10})?



Regularized Linear Regression

- Cost Function

$$J(\boldsymbol{\theta}) = \frac{1}{2n} \sum_{i=1}^n \left(h_{\boldsymbol{\theta}} \left(\mathbf{x}^{(i)} \right) - y^{(i)} \right)^2 + \frac{\lambda}{2} \sum_{j=1}^d \theta_j^2$$

- Fit by solving $\min_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$

- Gradient update:

$$\frac{\partial}{\partial \theta_0} J(\theta)$$

$$\theta_0 \leftarrow \theta_0 - \alpha \frac{1}{n} \sum_{i=1}^n \left(h_{\boldsymbol{\theta}} \left(\mathbf{x}^{(i)} \right) - y^{(i)} \right)$$

$$\frac{\partial}{\partial \theta_j} J(\theta)$$

$$\theta_j \leftarrow \theta_j - \alpha \frac{1}{n} \sum_{i=1}^n \left(h_{\boldsymbol{\theta}} \left(\mathbf{x}^{(i)} \right) - y^{(i)} \right) x_j^{(i)} - \lambda \theta_j$$

Regularized Linear Regression

$$J(\boldsymbol{\theta}) = \frac{1}{2n} \sum_{i=1}^n \left(h_{\boldsymbol{\theta}} \left(\mathbf{x}^{(i)} \right) - y^{(i)} \right)^2 + \frac{\lambda}{2} \sum_{j=1}^d \theta_j^2$$

$$\begin{aligned} \theta_0 &\leftarrow \theta_0 - \alpha \frac{1}{n} \sum_{i=1}^n \left(h_{\boldsymbol{\theta}} \left(\mathbf{x}^{(i)} \right) - y^{(i)} \right) \\ \theta_j &\leftarrow \theta_j - \alpha \frac{1}{n} \sum_{i=1}^n \left(h_{\boldsymbol{\theta}} \left(\mathbf{x}^{(i)} \right) - y^{(i)} \right) x_j^{(i)} - \lambda \theta_j \end{aligned}$$

- We can rewrite the gradient step as:

$$\theta_j \leftarrow \theta_j (1 - \alpha \lambda) - \alpha \frac{1}{n} \sum_{i=1}^n \left(h_{\boldsymbol{\theta}} \left(\mathbf{x}^{(i)} \right) - y^{(i)} \right) x_j^{(i)}$$

Linear regression with Multiple variables